

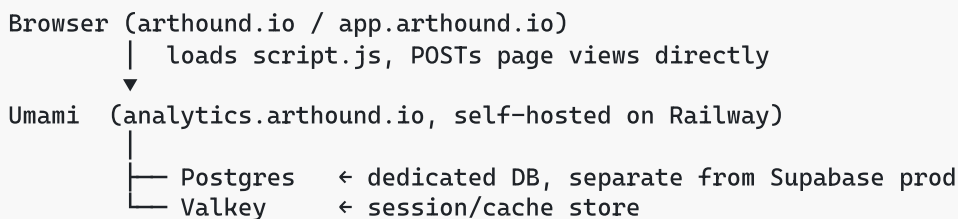
Web Analytics (Umami)

Last updated: 2026-06-03

ArtHound measures real visitor traffic to both public surfaces — the marketing landing page (`arthound.io`) and the app (`app.arthound.io`) — with a **self-hosted Umami** instance. Umami was chosen over an in-house FastAPI collector and over hosted Plausible/PostHog because it is cookieless, privacy-first, and keeps all visitor data on infrastructure we control, which matters when the people browsing the app are studios whose work is unreleased IP.

The tracker is **client-side only**. The FastAPI backend is not in the data path: the browser loads a script from the Umami instance and reports page views directly to it. This is deliberate — it means the landing page (a separately deployed static site that FastAPI never serves) and the app are measured the same way, through one dashboard, without coupling analytics to the application server.

Architecture



Hosting. Umami runs on Railway at `analytics.arthound.io`, with its **own dedicated Postgres and Valkey**, provisioned separately from the Supabase production database. The separation is intentional: analytics data and client production data never share a store. (See `[[project_production_deploy]]` for the broader Railway/Supabase split.)

No raw IPs are stored. Umami derives a daily visitor hash from `IP + user-agent + a rotating daily salt + website-id` and then discards the raw IP. There is no per-user IP log to query — by design. A consequence worth knowing: "unique visitors" is effectively *unique-per-day*, because the salt rotates each day, so the same person on two days counts as two uniques.

The two tracked sites

One Umami instance hosts two "websites", each with its own ID. Same `script.js` source, different `data-website-id`:

Surface	Host	Website ID	Snippet location
Landing	arthound.io	12c3bcbb-66d8-4292-af11-b228003f9159	arthound-landing/src/pages/index.astro (<head> , is:inline)
App	app.arthound.io	71177915-1f1e-4776-b9fd-3409a244f34c	frontend/index.html (<head>)

These IDs are not secrets — they are visible in page source on any visit — so they live in the repos in plaintext.

The snippet, and why it is hostname-gated

Rather than dropping Umami's stock `<script defer src=... data-website-id=...>` tag in directly, each site injects the tracker from a tiny inline guard that **only fires on the real production host**:

```
<!-- Umami analytics - only loads on the production host (keeps localhost/preview out of sto
<script>
  if (location.hostname === "app.arthound.io") {           // landing checks "arthound.io"/"www
    var s = document.createElement("script");
    s.defer = true;
    s.src = "https://analytics.arthound.io/script.js";
    s.setAttribute("data-website-id", "71177915-1f1e-4776-b9fd-3409a244f34c");
    document.head.appendChild(s);
  }
</script>
```

The guard means local dev (`localhost:5173`), Vite preview builds, and any non-production host never load the tracker, so the dashboard reflects real visitors only and is never polluted by our own development traffic. On the landing site the inline script is marked `is:inline` so Astro leaves it unbundled in the static output; the app's `index.html` is the Vite entry document and ships the snippet as-is.

SPA route changes are tracked automatically. The app is a single-page React app, so most navigation never reloads the page. Umami's tracker hooks the History API and records each client-side route change as a page view without any per-route instrumentation on our side.

Bots, crawlers, and traffic cleanliness

The dashboard is a *real-human* signal, not a raw-request signal, for two reasons that stack:

1. **It's a JavaScript tracker.** The large majority of crawlers, scrapers, uptime pingers and security scanners never execute JavaScript, so they hit the server but never fire the tracker and never count. This is the main reason the numbers are cleaner than raw Railway request logs.
2. **User-agent bot filtering.** For bots that *do* run JS, Umami matches the user-agent against a known-bot list server-side and drops them before recording. This is automatic; there is no toggle.

Combined with the hostname guard (which keeps our own dev/preview traffic out), the dashboard already approximates real visitors with no further configuration. The only residual noise is a headless browser deliberately spoofing a normal user-agent, a minority that usually shows up as an odd spike or junk referrer and can be filtered in the dashboard.

Excluding your own production visits. Because there is no IP, you cannot exclude yourself by IP. Use Umami's per-browser opt-out instead — run this once in the console on each site, on each machine you browse from:

```
localStorage.setItem('umami.disabled', 1);
```

That browser stops being counted on that origin.

Server-side / identified events (not wired)

Everything above is anonymous page-view tracking. If we ever need auth-correlated product events (e.g. "this studio did X"), Umami exposes two mechanisms:

- `umami.identify( ... )` — attach our own user/studio/vendor identity to the current browser session, callable from the app once authenticated (a natural spot is `AuthContext`).
- `POST /api/send` — report events server-side from FastAPI, the one path that *would* put the backend in the analytics data flow.

Neither is wired up. They are listed here so the next person knows the seam exists. Note that turning either on changes the privacy posture (it links analytics to identity), so it is a deliberate decision, not a default.

Deployment

The snippet edits ship with the normal release flow — there is no separate analytics deploy.

- **Landing** (`arhound-landing`): a static Astro site deployed on its own; pushing the edited `index.astro` to its production branch publishes it.
- **App** (`frontend/index.html`): the production Docker build runs `npm run build`, so the source `index.html` edit is baked into the served `frontend/dist`. The committed `frontend/dist` is `.dockerignore` d and stale — production always uses the fresh build. Promote through the usual dev→main→prod merge flow (see [\[\[project_production_deploy\]\]](#)).

Both snippets went live 2026-06-03.

Code Map

Concern	Location
Landing tracker snippet	<code>arthound-landing/src/pages/index.astro</code> (<code><head></code> , <code>is:inline</code>)
App tracker snippet	<code>frontend/index.html</code> (<code><head></code>)
Umami instance	Railway · <code>analytics.arthound.io</code> (own Postgres + Valkey)

Status & Non-Goals

Live: anonymous page-view tracking on both surfaces, hostname-gated, SPA route changes auto-tracked.

Explicit non-goals (current): - No raw IP storage or IP-level visitor inspection — Umami discards IPs by design. - No `umami.identify()` / auth-correlated events — the app does not tag sessions with `studio/vendor/user`. - No server-side `POST /api/send` events — FastAPI stays out of the analytics data path. - No custom event instrumentation (button clicks, funnels) beyond automatic page views.